

流水线与指令级并行

多条指令怎样合法重叠而不破坏顺序语义

408 · 计算机组成原理 Topic CO-04

母问题：怎样重叠执行，同时保持与 ISA 顺序轨迹等价？

CO-03 已给出一条指令内部的值依赖和提交边界。CO-04 把多条指令放到同一时间轴上，生成链是

Instruction Dependencies → Stage / Resource Occupancy → Need-Ready Constraints → Legal Overlap → For

箭头表示约束求解顺序：先承认程序依赖和资源事实，再判断哪些重叠合法；不是先背“停几拍”，再倒推理由。

本册学习范围

- 本册解释怎样把多条指令放进同一时间表，并在资源冲突、数据依赖和分支改变时保持程序顺序语义。
- 本册重点解释流水级、延迟、吞吐量、三类冒险、转发、停顿、冲刷、CPI 和精确异常。
- 乱序执行、超标量、多线程和多核只作为扩展例子；分析仍从“值何时产生、消费者何时需要、结果何时提交”开始。

1 术语先行：先把时间表中的词说清楚

本册的九个关键词

- **流水级**：把一条指令的组合逻辑切开后形成的一段工作，例如取指、译码、执行、访存和写回。
- **延迟 latency**：一条指令或一次操作从开始到结果可用所需的时间。
- **吞吐量 throughput**：流水线稳定运行后单位时间完成的指令数；它不等于单条指令延迟。
- **冒险 hazard**：下一拍本来想推进，但数据、硬件资源或下一条 PC 的条件还不满足。
- **RAW/WAR/WAW**：读后写、写后读、写后写三种寄存器依赖；其中 RAW 是“后继要读而前驱还没产生”的真实数据等待。
- **转发 forwarding**：把已经产生但尚未写回寄存器堆的值直接送给消费者。
- **停顿 stall**：冻结部分流水级，等待资源或数据满足条件。
- **冲刷 flush**：清除错误路径或异常路径上的年轻指令，防止它们留下软件可见副作用。
- **CPI/ILP**：CPI 是平均每条指令占用的时钟周期数；ILP 是同一指令流中可同时执行的独立指令数量。

目录

1 术语先行：先把时间表中的词说清楚	1
2 流水化从哪里生成	3
2.1 朴素方案与失败	3
2.2 吞吐率与延迟	3
3 时间模型：Produced、Ready、Need 与 Commit	3
4 Stage 与时空图：先声明模型	3
5 三类 Hazard：三种约束冲突	4
5.1 结构冲突	4
5.2 数据冲突	4
5.3 控制冲突	4
6 三种控制动作：Forward、Stall、Flush	5
7 贯穿母例：LOAD 后立即 ADD	5
8 控制冒险与预测状态机	5
8.1 惩罚从决策位置生成	5
8.2 两位饱和计数器	6
9 CPI 与总时间：从时空图计数	6
10 精确异常与提交顺序	6
11 代价、边界与 ILP 扩展	6
12 Flynn 与多处理器：并行对象先分清	7
13 五列概念边界	7
14 做题控制协议	7
15 本册与单指令路径的接口	8
16 考纲映射与一页压缩	8
17 边界说明：哪些结论必须依赖实现条件	9
18 补充细节：把流水线看成有约束的时间表	9

18.1 为什么常见五级划分只是一个可行模型	9
18.2 结构冒险：资源冲突先于“相关性”	9
18.3 数据冒险：依赖是静态关系，hazard 是时间冲突	9
18.4 控制冒险与分支预测状态机	10
18.5 流水线性能：吞吐量、延迟和总时间分开	10
18.6 动态流水、硬件多线程与多核：并行对象必须先分类	10
18.7 异常和中断穿过流水线时的精确性	11
18.8 模拟器与时空图的验证用法	11
19 扩充后的陌生流水线复原	11
20 补充细节：overview 增量：把性能指标放回 Cost 坐标	12
20.1 Amdahl：局部加速的整体上限	12
20.2 数量级与单位的防错	12
20.3 并行扩展的细节边界：线程、核心与共享内存	13

2 流水化从哪里生成

2.1 朴素方案与失败

若一条指令依次经历多个功能阶段，非流水实现会让当前未使用的部件闲置。把长组合路径切成 k 段，并在段间加入流水寄存器后，不同指令可以占用不同阶段。

设各段组合延迟为 T_i ，流水寄存器开销为 T_{reg} ，则

$$T_{clk} \geq \max_i T_i + T_{reg}.$$

理想情况下 n 条指令总时间为

$$T_{pipe} = (k + n - 1)T_{clk},$$

但这只是无 hazard、无 miss、段延迟均衡的成本模型。

2.2 吞吐率与延迟

流水线主要提高稳态吞吐率，不保证降低单条指令 latency。更深的流水可能缩短 T_{clk} ，却增加流水寄存器开销、填充/排空、分支惩罚和旁路复杂度。因此比较性能必须回到

$$T_{CPU} = IC \times CPI \times T_{clk}.$$

3 时间模型：Produced、Ready、Need 与 Commit

对一个值 v ，必须至少标四个时刻：

- **Produced**: 功能部件已经计算出 v ；
- **Ready**: 从某条实际路径看，消费者能够获得 v ；
- **Need**: 消费者所在阶段必须使用 v ；
- **Commit**: v 或其他效果写入软件可见架构状态。

RAW 依赖是否需要停顿取决于

$$t_{ready}(producer, path) \leq t_{need}(consumer).$$

若不等式失败，必须改变时间 (stall) 或改变路径 (增加 forwarding)；若值尚未 produced，任何旁路都无能为力。

Ready 不等于 Commit

forwarding 可以让 ALU 结果在写回前被下一条指令使用，但寄存器堆此时可能仍是旧值。可用时刻回答“谁能拿到”，提交时刻回答“软件可见状态何时改变”。

4 Stage 与时空图：先声明模型

经典五级教学模型常写为 IF、ID、EX、MEM、WB，但停顿数依赖以下假设：

- 各 stage 的真实职责与输入/输出；
- 寄存器堆同周期写/读的先后关系；
- 指令存储与数据存储是否共享端口；
- 分支在哪一级比较并产生目标；
- forwarding 的起点和终点；
- memory 是否固定一拍返回。

周期	IF	ID	EX	MEM	WB
1	I_1				
2	I_2	I_1			
3	I_3	I_2	I_1		
4	I_4	I_3	I_2	I_1	
5	I_5	I_4	I_3	I_2	I_1

时空图不是装饰：每个格子同时表示 stage、资源占用和流水寄存器边界。发生 stall 时必须说明哪些前级保持、哪个控制包被清零为 bubble、哪些后级继续推进。

5 三类 Hazard：三种约束冲突

5.1 结构冲突

同一周期两项工作需要同一不可并用资源，例如 IF 与 MEM 共用单端口存储器。解决路径只有三类：复制/增加端口，以空间换吞吐；错开调度，以周期换资源；或改变组织。是否冲突由题设资源图决定，不能由“无数据依赖”推出无冲突。

5.2 数据冲突

- **RAW**：消费者读生产者将写的新值，是真依赖；
- **WAR**：较年轻指令写得过早，破坏较老指令尚未完成的读；
- **WAW**：两个写同一目标的提交次序颠倒。

在单发射、按序发射、按序读写的简单五级流水中，通常只有 RAW 形成实际 hazard；WAR/WAW 需要乱序读写、多发射或不同完成延迟才可能出现。这个结论必须绑定模型，不能写成所有流水线的定律。

5.3 控制冲突

分支方向/目标尚未确定时，前级已经取入后续指令。等待、静态/动态预测、延迟槽和尽早决策都在交换平均停顿、硬件/编译器复杂度和错误路径成本。无论策略如何，错误路径效果不能提交。

6 三种控制动作：Forward、Stall、Flush

动作	保留/推进什么	阻止什么	使用条件
Forward	正常推进并改用旁路值	消费旧值	值已经 produced，且实际路径可达 need 端
Stall	保持 PC/前级状态或局部队列	未准备好的消费者前进	need 早于所有 ready 路径，或资源冲突
Flush	保留正确较老状态	错路/异常后的年轻效果提交	分支误判、异常、重定向

forwarding 不是“打开一个总旁路”。ALU → EX、load data → EX、result → branch compare、store data → MEM 是不同路径；题目没给路径，就不能假设存在。

7 贯穿母例：LOAD 后立即 ADD

$I_1 : \text{LOAD } R1, 0(R2), \quad I_2 : \text{ADD } R3, R1, R4.$

在经典五级模型中： I_1 的 EA 在 EX 产生，数据通常在 MEM 末尾产生； I_2 在自己的 EX 开始需要 $R1$ 。若相邻发射，数据的 ready 晚于 need，必须至少让消费者后移，直到存在可达旁路。

	1	2	3	4	5	6	7
I_1	IF	ID	EX	MEM	WB		
I_2		IF	ID	stall	EX	MEM	WB

该“一拍”只在上述 stage、memory latency、寄存器时序和 MEM-to-EX 旁路假设下成立。若分级更深、memory 多周期或旁路端点不同，必须重新比较 need/ready。

母例故意暴露的错误路径

“有 forwarding，所以永不停顿”忽略了值尚未产生；“LOAD 在 EX 算出了结果”把 EA 当成 load data；“ I_1 已经转发，所以已经完成”把 ready 当 commit。

8 控制冒险与预测状态机

8.1 惩罚从决策位置生成

若分支在第 d 级末尾才确定，错误路径已经进入的年轻指令数量由前端组织和预测时机决定。penalty 不是“分支固定几拍”，必须从决策级、已推进阶段和 flush 范围数出。

平均 CPI 贡献可写为

$$CPI_{branch} = f_{branch} r_{mispredict} P_{mispredict},$$

其中 $P_{mispredict}$ 必须是额外周期；若题目给的是总分支周期，需先剥离理想基线，避免重复计数。

8.2 两位饱和计数器

两位预测器是四状态 FSM，单次相反结果只使状态向另一方向移动一步。题目必须给初态、实际 T/NT 序列和采用高位/状态预测规则；循环“通常只错一次”不是无条件定律，初态和连续多轮会改变结果。

9 CPI 与总时间：从时空图计数

实际 CPI 可以分解为

$$CPI_{actual} = CPI_{ideal} + CPI_{struct} + CPI_{data} + CPI_{control} + CPI_{memory} + \cdots$$

分项只有在互斥或题设明确时才能直接相加；若一个 memory miss 同时阻止了后续依赖，不能把同一停顿重复归入两项。

有限指令序列的总周期应从时空图第一拍数到最后一条的规定完成/提交点。填充与排空已经包含在图中，不应再机械追加 $k - 1$ 。

10 精确异常与提交顺序

精确状态要求：异常指令之前的较老指令效果已经成立；异常指令和更年轻指令的禁止副作用没有提交。顺序流水可以冻结/清零写使能并 flush；更复杂实现把 produced/finished 与 ordered commit 分开。

execution may overlap $\wedge$ architectural trace remains recoverable

这条不变量连接 CO-03 的单指令 Write Set、CO-04 的多指令年龄顺序和 X-B01 的异常入口。OS handler 不在本册展开。

11 代价、边界与 ILP 扩展

机制	换来的能力	新成本	最小失败条件
更深流水	更短候选时钟、更多重叠	寄存器开销、分支/依赖周期数增加	最慢段/寄存器开销不降
Forwarding	减少已产生值的 RAW stall	比较器、MUX、布线和时序压力	值尚未产生或端点不可达
分支预测	降低平均控制停顿	预测状态、能耗、错误 flush	低可预测性 or 高 penalty
多发射	峰值 IPC 大于 1	端口、依赖检查、带宽	指令不独立 or 资源不足
VLIW	编译期显式并行、简化部分硬件	代码密度、调度和兼容性压力	动态延迟/并行不足

超标量、乱序、寄存器重命名、ROB、VLIW 和多核属于“同一依赖/资源/提交框架怎样扩张”的 Extension。408 主干只保留能解释 WAR/WAW、动态调度和 ordered commit 的最小接口，不把现代实现参数写成固定规律。

12 Flynn 与多处理器：并行对象先分清

流水线和超标量主要是在一条指令流内重叠；多处理器则增加执行上下文或指令流。按指令流/数据流分类：

类别	指令流/数据流	典型结构	题面判据
SISD	单/单	单核顺序处理器	一条指令处理一个数据序列
SIMD	单/多	向量机、向量指令	同一控制对多个数据元素执行
MISD	多/单	少见的专用容错结构	多条指令作用于同一数据流
MIMD	多/多	多核、SMP、通用多处理器	不同核心可执行不同线程/程序

向量处理属于 SIMD 的数据级并行；多核/SMP 属于 MIMD 的共享或分布式执行资源，不能把“多发射每拍多条”直接说成“多核”。硬件多线程则是在一个核心内保存多个线程上下文，在某线程等待或资源空闲时切换发射；它提高资源利用率，不自动增加物理 ALU 数量。

多处理器题仍沿本册三条约束检查：每个执行流的依赖与 ready/need、共享 Cache/总线/内存的资源冲突、最终架构提交是否符合各自 ISA 顺序。题目若进一步问 Cache 一致性、锁或线程调度，应转到相应专题；本节只确定并行层次与硬件接口。

并行层级的停止条件

先回答“增加的是数据元素、同一指令流中的发射宽度、线程上下文，还是物理核心/指令流”；再检查共享资源与提交边界。若无法指出新增并行对象，就不能仅凭“同时”判定属于 SIMD、超标量或多核。

13 五列概念边界

概念 A	≠ 概念 B	真正区别与题目信号	混淆后果
Latency	≠ Throughput	单条穿越时间 vs 单位时间完成数	误称流水必降单条延迟
依赖	≠ Hazard	程序语义关系 vs 当前时间/资源下的冲突	把所有 RAW 都算固定停顿
Produced	≠ Ready	已算出 vs 通过实际路径可被消费者拿到	假设不存在的旁路
Ready	≠ Commit	消费者可用 vs 架构状态已更新	精确异常和寄存器值判断错误
Stall	≠ Flush	等待后继续同一路径 vs 作废错误/年轻状态	保留了不应提交的控制包
ILP	≠ 多核并行	单指令流内部重叠 vs 多核心/任务并行	把 OS 调度和 Cache 一致性塞入流水线题

14 做题控制协议

1. 写 stage 定义、每级资源、寄存器读写时序、memory latency 和已有 forwarding；
2. 逐条列源/目的寄存器和控制转移，标 producer、consumer；

3. 对每个相关值标 produced、ready、need、commit;

4. 先画无冲突时空图，再叠加结构、数据和控制约束;

5. forwarding 必须写起点、终点和到达时刻; 剩余 ready-need 差才用 stall;

6. 分支写预测状态、决策级和 flush 范围;

7. 从最终时空图统计总周期/CPI，不重复添加填充或同一停顿;

8. 用年龄顺序和 Write Set 检查错误/异常路径没有提交。

压缩成两问

流水线题先问：这个值沿现有路径什么时候 ready，消费者什么时候 need？这个副作用在哪个年龄/时钟边界才允许 commit？

15 本册与单指令路径的接口

从单条路径到多条时间表

本册给每个值标出 produced（产生）、ready（沿实际路径可取得）、consumer need（消费者需要）和 architectural commit（写入软件可见状态）的时刻，再据此判断合法重叠、转发、停顿、冲刷和提交顺序。阶段名称不能替代这些时刻。

吞吐量是稳定填满后的单位时间完成量, latency 是单条指令从进入到结果/提交的时间; 两者和 clock rate、CPI、分支 penalty、miss stall 一起才形成 Cost。流水线加深或并行度增加不自动降低完整 CPU time。

16 考纲映射与一页压缩

传统考点	本册机制位置	下游接口
流水线性能	stage 延迟、填充/排空、CPU time	Rules 性能工具箱
结构/数据/控制冒险	资源、need/ready、Next PC	CO-03/CO-B01
转发、停顿、冲刷	三种控制动作与接口条件	具体时空图题
分支预测	预测 FSM、决策级、mispredict penalty	CO-I01
超标量/乱序/VLIW	ILP Extension 与按序提交边界	不扩张为多核 Topic

一句话

流水线把多条指令放到同一时间轴上：先用 need/ready 和资源约束判断哪些重叠合法，再用 forward、stall、flush 修复冲突，最终只允许符合年龄顺序和 ISA 的效果提交。

自测：为什么 forwarding 不能消灭所有 RAW？为什么 WAR/WAW 在简单五级顺序流水中通常不形成 hazard？为什么更深流水不保证更快？为什么分支 penalty 必须从决策级和 flush 范围数出？

17 边界说明：哪些结论必须依赖实现条件

经典五级流水、某个寄存器堆的半拍读写规则、现代处理器的级数/发射宽度和“所有 RISC/CISC 性能相当”等说法都必须带实现条件。Flynn 分类、多核、乱序和编译器后端只用于说明并行层次，不替代本册的 need/ready/commit 计算。

18 补充细节：把流水线看成有约束的时间表

流水线基础、性能、冒险、异常/中断和多处理器问题，都可以统一回接为

Instruction Stream $\rightarrow$ Stage Resources $\rightarrow$ Produced / Ready / Need $\rightarrow$ Forward / Stall / Flush $\rightarrow$ Ordered Comm

流水线不是“把一条指令分成五步后每条只需一个周期”，而是一个受资源和状态依赖约束的时间表。每个表格格子都应能回答：这条指令此刻占用什么资源、产生什么值、哪些后继可以消费、异常时哪些结果仍不可见。

18.1 为什么常见五级划分只是一个可行模型

经典 IF、ID、EX、MEM、WB 划分来自一组具体假设：取指、译码/寄存器读、算术逻辑、数据存储器访问和寄存器写回的组合延迟大致可分开，段间寄存器隔离各级状态。阶段数不是 ISA 事实；可以合并、拆分或复制阶段，只要保持语义等价和时序约束。段间寄存器带来额外建立/保持时间、填充与排空周期，也可能改变分支和数据依赖距离。

若各阶段延迟分别为 t_i 、流水寄存器开销为 t_r ，理想时钟周期近似为

$$T_{clk} = \max_i(t_i) + t_r.$$

一条指令的 latency 约为阶段数乘 T_{clk} ，稳态 throughput 约为每周期一条，但实际总周期还要加入填充、排空、结构冲突、数据停顿、分支冲刷和 Cache/存储等待。

18.2 结构冒险：资源冲突先于“相关性”

结构冒险发生在同一周期需要同一个不可复制资源，例如单端口存储器同时取指和访问数据、单总线同时需要两个驱动、单写口寄存器堆同拍需要多次提交。解决路径只有几类：复制资源、增加端口、改变阶段调度、让一条指令 stall，或把资源使用改成可分离事务。题目问“为什么某周期必须停”，先画资源占用表，而不是先找寄存器依赖。

18.3 数据冒险：依赖是静态关系，hazard 是时间冲突

RAW、WAR、WAW 先描述程序中的读写依赖；只有当实际生产/消费时刻和资源调度造成冲突，才形成 hazard。简单顺序五级流水通常只暴露 RAW，因为读在前、写在后且顺序提交；乱序或更深流水可能出现 WAR/WAW，需要重命名、保留站或提交队列等额外机制。不能把所有“有依赖”都直接换成固定停顿数。

对每条相关边写四个时刻：

producer produced path-ready consumer-need.

若 path-ready 早于或等于 need，可以 forwarding；若晚于 need，至少 stall 到可达。寄存器堆半拍读写、memory latency 和 forwarding 端点属于题设参数，不能从“经典五级”自动补齐。

动作/问题	保存什么	作废什么	判据
forward	已产生且可达的值	不改变年龄顺序	producer 的 path-ready 不晚于 consumer-need
stall	PC/某些流水寄存器保持，插入 bubble	不作废正确路径	等待资源或值变为可用
flush	年轻指令控制包清零/标无效	作废错误路径或异 常后的年轻状态	Next PC 已确定且错误路径不能提交

18.4 控制冒险与分支预测状态机

分支冒险的代价由三个量生成：预测何时做、真实决策在哪一级完成、前端已经进入了多少错误路径指令。预测正确时只需继续或选择目标；预测错误时冲刷从预测点到决策点之间的年轻指令，并从正确 Next PC 重新取指。若题目给两位饱和计数器，其四状态通常按弱/强不跳与弱/强跳形成，正确结果向强方向移动，错误结果向另一方向移动；计数器状态不是“分支本身已执行”的架构状态。

固定“分支停两拍”只有在决策级、预测策略、前端深度和 flush 范围都被固定时才成立。分支目标生成、条件判断和异常优先级可能在不同阶段，必须从题设图重新数。

18.5 流水线性能：吞吐量、延迟和总时间分开

理想 n 段流水执行 m 条独立指令，常见总周期为 $n + m - 1$ ；但这只是无停顿、每段单周期、无结构冲突的上界。实际可写成

$$Cycles \approx IC + Fill + Drain + \sum Stall + \sum Flush + \sum Miss\ Penalty,$$

再用 $CPU\ time = Cycles \times T_{clk}$ 或 $IC \times CPI \times T_{clk}$ 统一比较。吞吐量是稳态每单位时间完成多少条，latency 是单条指令从进入到结果/提交的时间；深流水可以改善频率和吞吐，却增加单条延迟、分支惩罚、依赖距离和寄存器开销。

超标量把同一周期可发射的独立指令数提高到大于一；多发射收益受真实 ILP、功能部件数量、寄存器端口和分支/Cache 等限制，不能把“理论发射宽度”直接当作实际 IPC。动态流水或乱序实现把 issue、execute、complete 和 commit 分开，结果可以先完成但仍须按年龄顺序提交，以保持精确异常。

18.6 动态流水、硬件多线程与多核：并行对象必须先分类

动态调度可以用保留站/评分表追踪操作数就绪和功能部件可用性，用重命名消除 WAR/WAW 的假依赖，再由 ROB 或等价结构按序提交。硬件多线程是在一个物理核心内保存多个线程上下文，交错发射以隐藏 cache miss 或长延迟，不等于增加 ALU 数量；多核则增加物理执行核心，进入 MIMD/共享内存层面的资源与一致性问题。

Flynn 分类只回答指令流/数据流的数量：SISD、SIMD、MISD、MIMD。SIMD/向量指令对多个数据元素使用同一控制，超标量是同一指令流的多条独立指令同拍发射，多核/SMP 是多个执行上下文或核心。题目出现“同时”时，先问新增的是数据元素、发射槽、线程上下文还是物理核心。

18.7 异常和中断穿过流水线时的精确性

同步异常由当前指令在某阶段产生，外中断由设备或外部事件提出；处理器可以在定义好的边界暂停前端、保存必要状态、选择入口并清空不能提交的年轻指令。故障类异常常允许修复后重新执行触发指令；陷阱、终止和外中断的返回点由 ISA 规定。关键不在“异常发生在哪一级”本身，而在：老指令哪些写回已提交、异常指令哪些副作用尚未提交、年轻指令哪些必须 flush、恢复后从哪一个 PC 重试。

异常题的三条线

同时画 value dependency、ready/need timing 和 exception/commit 三条线。只画流水级名称，无法判断某个寄存器写回是否已经可见，也无法判断异常处理后是重试当前指令还是从下一条继续。

18.8 模拟器与时空图的验证用法

流水线时空图或模拟器应按“预测—操作—回读”使用：

1. 先不打开处理机制，记录无 forwarding/stall 时第一次冲突出现在哪一格；
2. 只打开 forwarding，观察 ready 端点改变而不是直接删除依赖；
3. 再打开 stall，记录哪一个 PC/流水寄存器保持、哪一个槽变成 bubble；
4. 最后注入分支错误、异常或长延迟存储，核对 flush 范围、恢复 PC 和提交顺序。

模拟器不是答案生成器；它的输出只有在能被‘producer/ready/need/commit’四时刻解释时才算模型证据。

19 扩充后的陌生流水线复原

遇到新阶段、新预测规则或新发射宽度时，按以下顺序：

1. 抄出每阶段资源、延迟、段间寄存器、读写时序和存储响应参数；
2. 画无冲突时空图，确定填充、排空、结果可用和提交点；
3. 对每条依赖标 producer、consumer、produced、path-ready、need、commit；
4. 独立叠加结构冲突、RAW/WAR/WAW、分支预测和异常优先级；
5. 对 stall 写“保持谁/插入什么”，对 flush 写“作废谁/从哪重取”；
6. 统计完整周期并用 $IC \times CPI \times T_{clk}$ 检查数量级；
7. 若问题进入多核共享 Cache、锁或操作系统调度，停止在本册并转到对应专题。

CO-04 扩充停止条件

读者能从陌生的阶段划分和预测规则重建时空图，解释一次 forwarding/stall/flush 的必要性，区分单条 latency 与稳态 throughput，并指出何时必须按序 commit 时，流水线机制层完成；具体缓存一致性、线程调度和编译器指令选择不在本册扩张。

20 补充细节：overview 增量：把性能指标放回 Cost 坐标

性能题先区分两个目标：**响应时间**是单个任务从提交到完成的墙钟时间，**吞吐率**是单位时间完成的工作量。多核、批处理或深流水可能提高吞吐，却不必然缩短单个任务延迟；调度和 I/O 等待也会影响响应时间，但不应混入只问用户 CPU time 的公式。

CPU 执行时间的统一表达为

$$T_{CPU} = IC \times CPI \times T_{clk} = \frac{IC \times CPI}{f}, \quad CPI = \frac{\text{总时钟周期}}{IC}.$$

这里 IC 受编译器、ISA 和程序语义共同影响， CPI 受微架构、指令混合、Cache/TLB miss、分支和流水停顿影响， T_{clk} 受关键路径、流水段划分、寄存器开销和实现工艺影响。三者不是可独立任意优化的旋钮：减少 IC 的复杂指令可能增加 CPI ，加深流水可能缩短 T_{clk} 却增加 flush、旁路和寄存器成本。

指标/量	衡量对象	常见公式或口径	不能直接推出
吞吐率	单位时间完成多少工作	work/time	不等于单任务响应时间
响应时间	一个任务的完整墙钟耗时	提交至完成，含排队/等待	不等于用户 CPU time
IPS/MIPS	每秒执行的指令数/百万指令数	$IPS = f/CPI$	不同 ISA 的一条指令工作量不可直接等价
FLOPS	每秒浮点操作数	由浮点操作计数定义	不能代表整数、I/O 或分支负载
CPU time	CPU 执行程序代码的时间	$IC \times CPI \times T_{clk}$	不含墙钟 I/O 等待和其他任务占用

20.1 Amdahl：局部加速的整体上限

若原程序中比例 p 的部分获得加速比 s ，其余部分不变，则整体加速比为

$$S_{overall} = \frac{1}{(1 - p) + p/s}.$$

当 $s \rightarrow \infty$ 时， $S_{overall} \leq 1/(1 - p)$ ；因此先判断未优化部分是否已构成上限，再决定是否继续堆硬件。若题目给出优化前后具体时间，先把“可优化部分”和“不可优化部分”分开，不要把局部 s 当成整体加速。流水线、Cache、DMA、SIMD 和多核都只能加速它们实际覆盖的那部分工作。

20.2 数量级与单位的防错

主频/时钟周期使用十进制频率单位：1 GHz = 10⁹ Hz；地址空间和容量的幂次常按二进制解释：1 KiB = 2¹⁰ B。MIPS 的“M”是每秒百万条指令，FLOPS 的“F”是每秒浮点操作；题面写 MB、MiB、MHz 或 MT/s 时必须先确认它描述的是容量、基础时钟还是有效传输次数。

基准程序是负载集合而非单一程序。不同程序可能给出相反的机器排序，综合评价可用算术平均、几何平均或按使用频率加权；针对某个 benchmark 的特化优化也可能使结果不能代表一般负载。因而“主频更高”“MIPS 更大”或“流水级更多”都只能作为局部证据，最终仍回到统一的完整工作量和时间口径。

性能题的停止条件

先声明问的是吞吐、响应时间、用户 CPU time 还是总线有效带宽；再列 IC 、指令混合/CPI、 T_{clk} 和等待/重叠假设；最后用 Amdahl 上限或量纲检查结果。若只得到局部加速比而没有说明覆盖比例，性能结论尚未闭合。

20.3 并行扩展的细节边界：线程、核心与共享内存

硬件多线程、超标量和多核都能让“更多工作同时推进”，但增加的对象不同：细粒度多线程可按周期轮换线程，粗粒度多线程通常在长延迟事件时切换，SMT 在同一周期从多个线程填充发射槽；它们共享同一物理核心的执行资源，不等于增加 ALU 数量。多核则增加物理执行核心，属于 MIMD 资源扩展。

共享内存多处理器还要区分：**UMA** 中各核心访问统一内存的延迟近似一致；**NUMA** 中延迟随目标内存节点而变，调度和数据放置会进入性能成本。SMP 的核心、Cache 和内存通过互连共享数据时，必须有一致性/同步协议保证不同核心对同一地址的可见性顺序；这不是单核 forwarding，也不能从“多核”自动推出某一种 MESI 状态机。题目只问 408 主干时，保留“共享副本需要一致性接口”这一边界，具体协议、锁和操作系统调度另行展开。

对象	增加的并行维度	共享什么	主要 Cost/边界
SIMD/向量	一条指令处理多个数据元素	控制流与向量执行资源	数据相关/分支会降低利用率
超标量	同一线程同拍发射多条指令	核心内功能部件/寄存器	依赖、端口和提交宽度限制
硬件多线程	多个线程上下文交错/同时发射	一个物理核心资源	隐藏 miss，不自动增加物理算力
多核/SMP	多个物理执行流	内存、互连和共享 Cache	一致性、同步、UMA/NUMA 延迟